

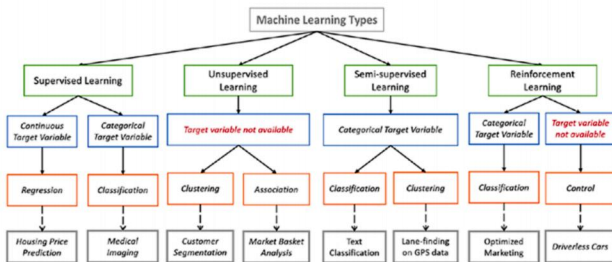
Machine Learning Architecture

Practical Principles of Logistic Regression, Trees, and Forests

Fenglin Liu, Chuyu Qiao, Letian Lin

The Big Picture: ML Architecture

- **Data Input:** Collecting and cleaning data (The "Fuel").
- **Learning Type Selection:**
 - **Supervised:** Learning from labeled examples (The Teacher).
 - **Unsupervised:** Finding hidden patterns (The Explorer).
- **Model Training:** Choosing the right "Brain" (Algorithm).
- **Evaluation:** Measuring error and stability (The Quality Control).



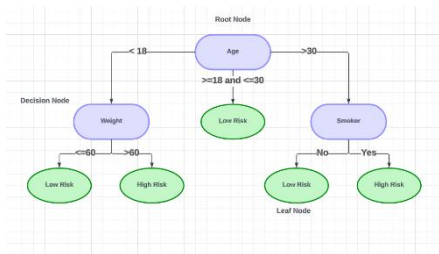
Decision Trees

- 决策树（**Decision Tree**）是一种基于树形结构的监督学习算法，通过递归划分特征空间，将复杂问题拆解为一系列简单的“条件判断”，最终实现分类（预测类别）或回归（预测连续值）。
- 优点：计算复杂度不高，输出结果易于理解，对中间值的缺失不敏感，可以处理不相关特征数据。
- 缺点：可能产生过拟合问题

Decision Tree Algorithms	Algorithms
	Classification and Regression Tree (CART)
	Iterative Dichotomiser 3 (ID3)
	C4.5 and C5.0 (different versions of a powerful approach)
	Chi-squared Automatic Interaction Detection (CHAID)
	Random Forest
	Conditional Decision Trees

Decision Trees: The Logic of Splitting

- **Tree Structure:** The result of these spatial splits is a hierarchical structure (Root → Nodes → Leaves).
- **White Box Logic:** Every prediction path is a set of human-readable "If-Then" rules.
- **Prediction:** A new data point follows the path based on its feature values until it reaches a leaf node (the final class).



创建分支的伪代码函数createBranch()如下所示:

```
检测数据集中的每个子项是否属于同一分类:
If so return 类标签
Else
    寻找划分数据集的最好特征
    划分数据集
    创建分支节点
    for 每个划分的子集
        调用函数createBranch并增加
    返回结果到分支节点中
    return 分支节点
```

- 划分数据集的大原则是: 将无序的数据变得更加有序。
- 在划分数据集之前之后信息发生的变化称为信息增益, 获得信息增益最高的特征就是最好的选择。

Implementing Decision Trees

Using the real world data to build a Classification and Regression Tree.

Purchase Preference			
Variable	Description	Code/Values	Variable Name
1	Unique Identifier of Customer	1 to 1201	CUSTOMER_ID
2	Choice of Product Purchased	1, 2,3 and 4	ProductChoice
3	Customer member reward points	1-13	MembershipPoints
4	The mode of payment by the customer	Cash, CreditCard ,etc	ModeOfPayment
5	Customer resident city	City Name	ResidentCity
6	Numberof months since the first purchase made by the customer	Positive Integer	PurchaseTenure
7	Channel of purchase	Online/Offline	Channel
8	Income of customer	1- Lowest ... 9 - Highest	IncomeClass
9	Buying propensity rating of the customer	VeryHigh, High, Medium, Low, Unknown	CustomerPropensity
10	Age of customer as on last purchase	Positive Integer	CustomerAge
11	Marital Status of customer	1- Married, 0- Single	MaritalStatus
12	Months since the last purchase	Positive Integer	LastPurchaseDuration

Pre-processing: Cleaning and Sampling

- **Feature Selection:** We filter only relevant predictors like `IncomeClass` and `MembershipPoints`.
- **Stratified Sampling:** To ensure the model learns all classes equally, we use `stratified()` to maintain the ratio of `ProductChoice`.

```
# Pre-processing
Data_Purchase <- read.csv("...your_document/Purchase Prediction Dataset.csv", header = TRUE)
head(Data_Purchase,3)
Data_Purchase <- Data_Purchase[,c("CUSTOMER_ID", "ProductChoice", "MembershipPoints", "IncomeClass", "CustomerPropensity",
"LastPurchaseDuration")]

Data_Purchase <- na.omit(Data_Purchase)
Data_Purchase$CUSTOMER_ID <- as.character(Data_Purchase$CUSTOMER_ID)

# Stratified sampling
#install.packages('splitstackshape')
library(splitstackshape)
#Data_Purchase_Model <-Data_Purchase[sample(nrow(Data_Purchase),10000),
Data_Purchase_Model <- stratified(Data_Purchase, group=c("ProductChoice"), size=10000, replace=FALSE)
table(Data_Purchase_Model$ProductChoice)
Data_Purchase_Model$ProductChoice <-as.factor(Data_Purchase_Model$ProductChoice)
Data_Purchase_Model$IncomeClass <-as.factor(Data_Purchase_Model$IncomeClass)
Data_Purchase_Model$CustomerPropensity <-as.factor(Data_Purchase_Model$CustomerPropensity)
```

Pre-processing: Train/Test Split

- **Purpose:** We train the model on one portion of data and "exam" it on unseen data to test its generalization.
- **Ratio:** A common split is **70% Training** and **30% Testing**.

```
# Set seed for reproducibility
set . seed(917)

# 1. Randomly sample 70% for Training
train_idx <- sample (nrow (Data_Purchase_Model),
                    size = nrow (Data_Purchase_Model) *
                        0.7)
train <- Data_Purchase_Model[train_idx , ]

# 2. Remaining 30% for Testing
test <- Data_Purchase_Model[-train_idx , ]
```

Tip: Always ensure the Test set contains no data points from the Training set to avoid "cheating."

CART: Classification and Regression Trees

- **Dual Nature:** CART can be used for regression and classification.
- **Impurity Measures:**
 - **Regression:** Uses *Residual Sum of Squares (RSS)* - sum of squared deviation about the mean.
 - **Classification:** Uses **Gini-Index** as the node impurity measure

Algorithm Pseudo-code

- 1 Start at the root node.
- 2 **Iterate:** For each attribute X , find subset S that **minimizes** the RSS of the two children.
- 3 **Select:** Choose the split providing the **maximum** information gain.
- 4 **Stopping Criterion:** Check if the relative decrease in impurity is below a prescribed threshold.
- 5 **Terminate:** If yes, stop splitting; otherwise, repeat Step 2.

CART Implementation in R

```
CARTModel <- rpart(ProductChoice ~ IncomeClass + CustomerPropensity + LastPurchaseDuration +  
  MembershipPoints, data=train)  
summary(CARTModel)
```

```
> summary(CARTModel)
```

Call:

```
rpart(formula = ProductChoice ~ IncomeClass + CustomerPropensity +  
  LastPurchaseDuration + MembershipPoints, data = train)  
n= 28000
```

	CP	nsplit	rel error	xerror	xstd
1	0.09654646	0	1.0000000	1.0064873	0.003439637
2	0.02380271	1	0.9034535	0.9034535	0.003734229
3	0.02065446	2	0.8796508	0.8876646	0.003768463
4	0.01000000	3	0.8589964	0.8670101	0.003809256

Variable importance

变量重要性百分比

CustomerPropensity	MembershipPoints	LastPurchaseDuration	IncomeClass
52	39	8	1

```
Node number 1: 28000 observations, complexity param=0.09654646  
predicted class=1 expected loss=0.7487143 P(node) =1  
class counts: 7036 6994 6973 6997  
probabilities: 0.251 0.250 0.249 0.250
```

Logistic Regression

假设现在有一些数据点，用一条直线对这些点进行拟合（该线称为最佳拟合直线），这个拟合过程就称作回归。

利用**Logistic**回归进行分类的主要思想：根据现有数据对分类边界线建立回归公式，以此进行分类。

- **Concept:** Used for binary outcomes (Yes/No, 0/1).
- **The Logit Link:** It transforms a linear combination of predictors into a probability using the **Sigmoid Function**.
- **Output:** Unlike linear regression, it outputs the **Log-Odds** of an event occurring.
- **When to use:** When you need to understand the influence of specific variables on the probability of an outcome.
 - 优点：计算代价不高，易于理解和实现
 - 缺点：容易欠拟合，分类精度可能不高
 - 适用数据类型：数值型和标称型数据

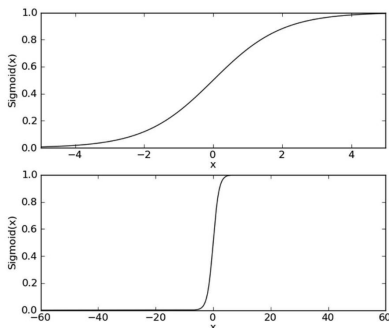
Logistic Regression: Why the Shift?

Limitations of Linear Regression:

- **Output Range:** Linear models can predict values < 0 or > 1 , which is invalid for probabilities.
- **Error Distribution:** Errors follow a **Binomial Distribution** rather than a Normal (Gaussian) one.

The Solution:

- Use a **Link Function** (Logit) to map linear inputs to a $(0, 1)$ scale.
- Specifically designed for **categorical** responses (e.g., Default vs. No Default).



两种坐标尺度下的Sigmoid函数图。

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Three Types of Logistic Regression

1. Binomial Logistic Regression

The response has only two levels (0/1, Success/Failure). *Example: Default vs. No Default, Disease vs. Healthy.*

2. Multinomial Logistic Regression

Three or more levels **without** inherent order. *Example: Predicting which of 3 different drugs a patient prefers.*

3. Ordered Logistic Regression

Three or more levels **with** a built-in order. *Example: Patient satisfaction scores (1-5) or Cancer Staging.*

Implementing Logistic Regression in R

The glm (Generalized Linear Model) function is the standard tool for this architecture.

```
train$ProductChoice_binom <- ifelse(train$ProductChoice ==1,1,0);
test$ProductChoice_binom <- ifelse(test$ProductChoice ==1,1,0);

glm_ProductChoice_binom <- glm(ProductChoice_binom ~ MembershipPoints
                               +IncomeClass +CustomerPropensity +LastPurchaseDuration,
                               data = train,
                               family = binomial(link="logit"))

summary(glm_ProductChoice_binom)
```

Implementing Logistic Regression in R

Call:
glm(formula, data = Customer, family = "logit"),

系数的最大似然估计值 (β),
对应logit(P)的线性项: $\text{logit}(P) = \beta_0 + \beta_1 \text{MP} + \beta_2 \text{C2} + \dots$, MP每增加1单位, $\text{logit}(P)$ (对数优势) 增加0.044, 正负号表示效应方向

系数估计的标准误差 (SE),
反映估计精度 (SE越小, 估计越稳定)

显著性水平, 如p值<0.05:
自变量与结局显著相关

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-1.901403	0.385707	-4.930	8.24e-07	***
MembershipPoints	0.044293	0.005793	7.645	2.08e-14	***
IncomeClass2	-0.536802	0.583824	-0.919	0.3579	
IncomeClass3	0.668610	0.389247	1.718	0.0859	.
IncomeClass4	0.424142	0.383156	1.107	0.2683	
IncomeClass5	0.467211	0.382420	1.222	0.2218	
IncomeClass6	0.341454	0.382647	0.892	0.3722	
IncomeClass7	0.317858	0.383482	0.829	0.4072	
IncomeClass8	0.193686	0.388640	0.498	0.6182	
IncomeClass9	0.501812	0.427067	1.175	0.2400	
CustomerPropensityLow	0.768436	0.054133	14.195	< 2e-16	***
CustomerPropensityMedium	0.461935	0.054991	8.400	< 2e-16	***
CustomerPropensityUnknown	0.977535	0.048603	20.113	< 2e-16	***
CustomerPropensityVeryHigh	-0.440972	0.066315	-6.650	2.94e-11	***
LastPurchaseDuration	-0.058637	0.003380	-17.349	< 2e-16	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 31570 on 27999 degrees of freedom
Residual deviance: 29703 on 27985 degrees of freedom
AIC: 29733

Number of Fisher Scoring iterations: 4

Model Evaluation: The Confusion Matrix

- **Definition:** A table used to describe the performance of a classification model on a set of test data for which the true values are known.
- **Key Components (Binary Case):**

		Predicted Class	
		Positive (1)	Negative (0)
Actual Class	Positive (1)	TP (True Positive) 抓对了	FN (False Negative) 漏抓了
	Negative (0)	FP (False Positive) 抓错了	TN (True Negative) 没抓错

- **Type I Error (FP):** "False Alarm" - Predicting 1 when it's actually 0.
- **Type II Error (FN):** "Missed Opportunity" - Predicting 0 when it's actually 1.

Model Evaluation: The Confusion Matrix

```
library(gmodels)
purchase_pred_train <- predict(CARTModel, train, type = "class")
CrossTable(train$ProductChoice, purchase_pred_train,
  prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE,
  dnn = c('actual default', 'predicted default'))
```

actual default	predicted default			Row Total
	1	3	4	
1	4179 0.149	1918 0.068	939 0.034	7036
2	3410 0.122	2651 0.095	933 0.033	6994
3	2450 0.087	3785 0.135	738 0.026	6973
4	1860 0.066	3109 0.111	2028 0.072	6997
Column Total	11899	11463	4638	28000

It looks like a poor model for this dataset. If you observe, the training model doesn't even predict any instance of class 3.

Quantitative Metrics: From Counts to Ratios

Using the counts from the Confusion Matrix, we calculate:

- ① **Accuracy:** Overall correctness.

所有人里，你
判断正确的占
多少？

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- ② **Precision:** How many predicted positives are actually positive?

在你抓的人里，
真正的小偷占
多少？

$$\text{Precision} = \frac{TP}{TP + FP}$$

- ③ **Sensitivity / Recall:** How many actual positives did we catch?

所有的坏人里，
你抓住了多少？

$$\text{Recall} = \frac{TP}{TP + FN}$$

- ④ **F1-Score:** The harmonic mean of Precision and Recall. (Useful for imbalanced data).

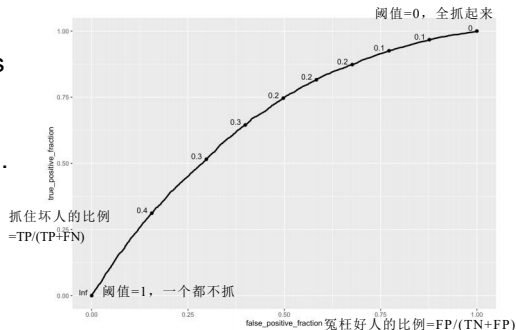
通过调和平均将Precision和Recall合并为一个指标

The ROC Curve and AUC

- **ROC** (receiver operating characteristic) **Curve** : Plots **True Positive Rate** (Sensitivity) vs. **False Positive Rate** (1-Specificity).

- **AUC** (Area Under Curve):

- **1.0**: Perfect model.
- **0.5**: No better than random guessing.
- **0.7-0.8**: Acceptable.
- **>0.9**: Excellent.



理想的ROC曲线形状是怎样的？
随机猜测的ROC曲线形状是怎样的？

The ROC Curve and AUC

- ROC Curve: Plots True Positive Rate (Sensitivity) vs. False Positive Rate (1-Specificity).
- AUC (Area Under Curve):
 - 1.0: Perfect model.
 - 0.5: No better than random guessing.
 - 0.7-0.8: Acceptable.
 - >0.9: Excellent.

```
# Fit a binary logistic model on the modified
# dependent variable, ProductChoice_binom.
glm_ProductChoice_binom <- glm ( ProductChoice_binom ~
  MembershipPoints + IncomeClass +
  CustomerPropensity + LastPurchaseDuration , data =
  train , family=binomial ( link = " logit " ))

summary ( glm_ProductChoice_binom )

library(pROC)
test_binom <- predict(glm_ProductChoice_binom,newdata=test, type="response")
roc_obj <- roc(response = test$ProductChoice_binom, predictor = test_binom,
  levels = c(0, 1), # 明确负/正类标签（默认按出现顺序，建议显式指定）
  direction = "<") # 若预测值越大越可能为阳性，用"<"（默认）；反之用">"

# 查看AUC及置信区间
auc(roc_obj) # 输出AUC值
# 绘制ROC曲线
plot(roc_obj, main = "ROC Curve (pROC)", col = "red", lwd = 2)
# 添加AUC文本
text(0.6, 0.2, paste0("AUC = ", round(auc(roc_obj), 2)), cex = 1.2)
# 高级绘图（带最佳阈值）
plot(roc_obj,
  print.auc = TRUE, # 图上显示AUC
  auc.polygon = TRUE, # 填充AUC区域
  auc.polygon.col = "lightblue", # 填充色
  print.thres = "best", # 显示最佳阈值（Youden指数最大处）
  legacy.axes = TRUE) # x轴为FPR（0~1），y轴为TPR（0~1）（默认）
```

也可以使用ROCR等R包

- 使用实验2中TCGA GDC Liver Cancer的表达量数据，选用合适的机器学习方法，对肿瘤/正常样本进行分类
 - 数据预处理与特征选择

提示：可选择Top50差异表达基因，或Top50方差最大的基因
 - 数据分割
 - 模型训练，简要说明为何选择这种方法
 - 模型评价
- 允许使用Vibe coding，必须说明用途及提示词，否则计0分。
- 命名为“学号_姓名_第X次_周Y班”，提交到教学网。

References

- Primary Source:

Textbook

Ramasubramanian, K., & Singh, A. (2019). *Machine Learning Using R: With Multi- Variate Prediction Models, Unsupervised and Supervised Learning* (2nd ed.). Apress.

(<https://link.springer.com/book/10.1007/978-1-4842-4215-5#accessibility-statement>)

Peter Harrington. 机器学习实战. 人民邮电出版社.

Thank you!